

KONSEP MIGRASI PERFORMA

RepoManager App — Expo SDK 51 → SDK 56 (New Architecture) + JSI File API

Disusun: 9 Agustus 2026 | Untuk: Zen (solo developer) | Status: **Konsep** — belum eksekusi

Dokumen ini merangkum rencana migrasi buat ngatasin 3 keluhan performa yang udah didiskusikan: **fetch ke GitHub lambat**, **batas waktu Dashboard kurang (bukan solusi rill)**, dan **pindah antar tab makan 2-3 detik**. Keputusan yang udah disepakati: pakai kombinasi **fix logic (hilangin kerjaan sia-sia)** + **fondasi JSI lewat New Architecture (SDK 56)** buat operasi yang emang gak bisa dihindari.

Arti warna di dokumen ini:

KRITIS

Resiko bikin app error/crash atau celah keamanan kalau salah kerjain. Wajib extra hati-hati & testing.

PERLU UJI

Breaking change tingkat sedang — kemungkinan bug ada, tapi gak sampai fatal kalau ketahuan lewat testing.

AMAN

Resiko rendah, perubahan terisolasi/reversibel dengan mudah.

WORKFLOW

Urutan kerja / tahapan proses — bukan soal resiko, tapi soal alur.

KEPUTUSAN

Hal yang udah final disepakati di diskusi sebelumnya, gak dibahas ulang.

1. LATAR BELAKANG & AKAR MASALAH

Kondisi saat ini

Expo SDK 51 / React Native 0.74 / Old Architecture. Isomorphic-git jalan lewat `fsAdapter.js` custom di atas `expo-file-system` legacy — tiap baca/tulis 1 git object (blob/tree/commit) = 1 round-trip async lewat bridge JS↔Native. Ini akar kenapa fetch, statusMatrix, dan operasi git lain kerasa lambat walau ukuran repo kecil.

3 keluhan yang jadi fokus

Keluhan	Akar masalah sebenarnya	Kategori fix
Fetch ke GitHub lambat, ada perubahan lokal	Bridge overhead per-object + walk commit 4x buat ahead/behind	KRITIS
Dashboard mepet batas waktu (25 detik)	statusMatrix scan penuh tiap load, timeout cuma nutup gejala	PERLU UJI
Pindah tab 2-3 detik	List repo nunggu status SEMUA repo keluar dulu baru render (blocking)	AMAN

Kenapa gak cuma naikin JSI doang / gak cuma fix logic doang

JSI ngebikin operasi yang MEMANG PERLU jalan lebih cepat (I/O). Tapi dia gak ngilangin operasi yang SEBENARNYA GAK PERLU dikerjain (misal fetch penuh buat itung ahead/behind, padahal GitHub API bisa jawab langsung). Dua lapisan masalah beda — makanya rencana ini gabungin keduanya, bukan pilih salah satu.

2. WORKFLOW Pengerjaan (BUKAN CUMA DAFTAR TASK)

Urutan di bawah ini DIRANCANG SUPAYA TIAP TAHAP BISA DI-TEST SENDIRI sebelum lanjut ke tahap berikutnya — bukan sekumpulan task yang dikerjakan sekaligus. Kalau tahap N gagal test, tahap N+1 BELUM BOLEH dimulai. Ini prinsip utama biar gak keulang kasus 'kerja sia-sia' kayak isomorphic-git kemarin.

FASE 0 Baseline & Rollback Point

Bikin branch/tag git terpisah dari kondisi sekarang yang jalan normal. Catat versi semua dependency existing (sudah di package.json). Ini adalah titik balik kalau migrasi gagal total — WAJIB ada sebelum nyentuh apapun.

FASE 1 Upgrade SDK bertahap (51 → 56)

Naik SATU MAJOR VERSION dulu (51→52), build dev-client, kirim APK, Zen test manual fitur-fitur inti (login, clone, lihat repo). Kalau lolos, lanjut 52→53, dst sampai 56. JANGAN loncat langsung ke 56 — kalau ada breaking change, susah dilacak dari versi mana asalnya kalau naik sekaligus 5 versi.

Kenapa bertahap bukan langsung: kalau ada 1 dependency native (react-native-screens, gesture-handler, get-random-values) yang rewel, jauh lebih gampang nemuin biang keroknya kalau upgrade satu-satu.

FASE 2 Aktifkan New Architecture & regresi penuh

New Architecture di SDK 55/56 gak bisa dimatiin. Setelah nyala, WAJIB regresi manual semua fitur existing dulu SEBELUM nyentuh fsAdapter.js sama sekali: Login OAuth, Clone, Upload (zip/file), Branch, Working Tree, Sync (push/pull), Merge/PR, Stash, Storage Manager. Tiap fitur dites pakai APK dev-client yang dikirim ke HP Zen langsung.

Ini fase paling beresiko di seluruh rencana — kalau ada yang rusak di sini, HARUS diberesin dulu sebelum lanjut, jangan ditumpuk sama perubahan fsAdapter (biar gampang nebak sumber bug-nya).

FASE 3 Ganti fsAdapter.js ke File/Directory API baru

Baru di fase ini isomorphic-git di-arahin ke API filesystem baru (berbasis JSI). Behavior readFile/writeFile/mkdir/stat dkk WAJIB dites identik hasilnya sama versi lama (bandingin isi file sebelum-sesudah clone/commit yang sama), bukan cuma "jalan tanpa error".

REPOS_ROOT dan struktur folder sandbox TIDAK diubah - supaya repo yang udah ke-clone sebelumnya di HP Zen tetap kebaca, gak perlu clone ulang semua.

FASE 4 Fix logic (hilangin kerjaan sia-sia)

Diterapkan DI ATAS fondasi baru yang udah stabil: (a) ganti hitung ahead/behind pakai GitHub Compare API buat commit yang udah ke-push, (b) update cache statusMatrix secara incremental abis commit (bukan invalidate total), (c) render list repo progresif per-item, gak nunggu semua repo kelar discan.

Ini scope-nya sama kayak yang udah disepakati sebelumnya di chat — cuma urutannya dipindah ke belakang supaya jalan di atas fondasi yang udah teruji, bukan numpuk sama resiko migrasi SDK.

FASE 5 Regresi akhir & serah terima

Testing menyeluruh sekali lagi (checklist sama kayak Fase 2) + ukur waktu nyata fetch/tab-switch sebelum-sesudah di HP Zen. HANDOFF.md diupdate biar sesi berikutnya tau app udah di New Architecture, jangan diasumsikan Old Architecture lagi.

3. PETA RESIKO DEPENDENCY (SECURITY & STABILITAS)

Sesuai aturan #3: kegagalan di titik-titik ini bisa bikin PROJECT ERROR TOTAL atau nyentuh sisi keamanan (token, auth). Bagian ini dipisah biar kelihatan jelas, bukan ketutup di daftar task biasa.

Dependency / Area	Resiko	Level	Penanganan
auth/githubAuth.js, authStore.js (OAuth token, SecureStore)	TIDAK disentuh sama sekali selama migrasi. Kalau ada kegagalan New Architecture di sini, token/secret bisa kena resiko kebocoran atau gagal decrypt.	KRITIS	Scope migrasi SENGAJA gak termasuk file ini. Kalau expo-secure-store butuh update demi New Architecture, itu dites terpisah, gak digabung sama perubahan fsAdapter.
react-native-get-random-values (dipakai isomorphic-git buat crypto)	Versi terbaru mensyaratkan minimal RN 0.81/SDK 54 — searah sama rencana upgrade, tapi kalau versi ke-skip pas upgrade bertahap, crypto bisa diam-diam gagal / fallback gak aman.	KRITIS	Cek versi exact-nya di tiap fase upgrade (Fase 1), jangan diasumsikan otomatis kompatibel.
react-native-screens	Ada known issue predictive-back gesture Android di New Architecture.	PERLU UJI	Ditest manual navigasi Android di Fase 2 sebelum lanjut.
react-native-gesture-handler	Perlu verifikasi ulang compatibility New Architecture.	PERLU UJI	Termasuk checklist regresi Fase 2.
metro.config.js (unstable_enablePackageExports, fix isomorphic-git)	Internal Metro berubah tiap major SDK — fix ini bisa jadi gak perlu lagi ATAU perlu disesuaikan ulang.	PERLU UJI	Divalidasi ulang tiap naik 1 SDK version di Fase 1, bukan diasumsikan tetap jalan.
logger.js — redactSecrets()	Kalau ada refactor gak sengaja yang lewatin fungsi ini, token bisa ke-log mentah di log aktivitas/debug.	KRITIS	TIDAK ada perubahan pada logger.js di seluruh rencana ini. Kalau muncul kebutuhan ubah, dibahas terpisah.
statusCache.js (arah bahaya cache satu sisi)	Fix incremental cache (Fase 4) kalau salah scope bisa bikin cache bilang "bersih" padahal berubah → resiko Pull nimpa data.	KRITIS	Scope incremental update DIBATASI KETAT cuma buat kasus commit (predictable). TTL 15 detik existing tetap dipertahankan sebagai jaring kedua.

4. TARGET PERFORMA (FETCH, TAB, DASHBOARD)

Sesuai aturan #2: memaksimalkan performa dengan bug seminimal mungkin. Angka di bawah ini ESTIMASI ARAH PERBAIKAN berdasarkan penyebab yang udah diidentifikasi, BUKAN janji angka pasti — waktu rill baru kekonfirmasi pas testing di HP Zen tiap fase.

Operasi	Sekarang	Sumber lambat	Setelah migrasi
Fetch + ahead/behind (commit sudah di-push)	~detik hingga puluhan detik	git.fetch() + walk commit 4x lewat bridge	Fase 4: 1 HTTP request GitHub API, hilangin fetch lokal sama sekali
Dashboard load (statusMatrix)	Mepet 25 detik utk repo besar	Scan semua file, bridge per-file	Fase 3 (JSI) mempercepat tiap I/O + Fase 4 (incremental) hilangin scan setelah commit
Pindah tab Local Repos	2-3 detik	Nunggu status SEMUA repo sebelum render	Fase 4: render langsung, badge status nyusul progresif
Clone / Pull repo besar	Lambat, sebanding jumlah file	Bridge overhead tiap object git	Fase 3 (JSI): tetap perlu scan penuh (gak bisa dihindari), tapi tiap I/O jauh lebih cepat

Catatan jujur: React Native (arsitektur apapun) tetap ada JS engine di tengah — gak akan pernah 100% identik kecepatan CLI Python di Termux yang langsung native. New Architecture + JSI adalah jalur PALING DEKAT ke situ yang realistis buat app ini, bukan solusi sempurna.

5. CARA ZEN TEST SENDIRI (DEV-CLIENT)

Tiap fase (terutama Fase 1 dan 2) menghasilkan build eas `build --profile development --platform android` yang dikirim sebagai APK ke Zen. Instal langsung di HP, connect ke Metro bundler seperti biasa. Checklist regresi minimal yang perlu dicoba tiap kali terima APK baru:

- Login OAuth Device Flow — sampai profile kebaca
- Buka Dashboard — panel status + Aksi Cepat tampil
- Clone repo baru — sampai selesai tanpa error
- Buka Local Repos — badge status Clean/Modified muncul
- Working Tree: Add file + Commit
- Push & Pull (kalau sudah masuk Fase 4, ke tahap ini)
- Upload (zip extract / file)
- Pindah antar 4 tab berkali-kali cepat — cek gak nge-freeze